Ders 5 Genişletilmiş Notu: Özyinelemeli Problemler ve Bağlı Listeler

Ders 5 Genişletilmiş Notu: Özyinelemeli Problemler ve Bağlı Listeler

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Asallık Testi Akış Diyagramındaki Düzeltme
- 2. Özyinelemeye Kısa Dönüş
- 3. Özyinelemeli Problem 1: İki Sayıyı Toplama ile Çarpmak
- 4. Özyinelemeli Problem 2: Rakamlar Toplamı
- 5. Özyinelemeli Problem 3: Palindrom Sayı
- 6. Özyinelemeli Problem 4: Asallık Testi
- 7. Alıştırma: Bir Dizinin r'li Kombinasyonları
- 8. Özyineleme Tasarımının Özeti
- 9. Diziden Bağlı Listeye
- 10. Bağlı Listeyi Gerçekleştirme Yolları
- 11. Gezinme, Arama ve Güncelleme
- 12. Ekleme (Insert) İşlemi
- 13. Bağlı Listelerin Kullanım Alanları
- 14. Silme (Delete) İşlemi
- 15. Bağlı Liste Türleri
- Örnek Sorular

1. Asallık Testi Akış Diyagramındaki Düzeltme

Hoca derse, önceki derste gösterdiği asal sayı bulma algoritmasındaki bir hatayı düzelterek başladı. Hatayı bir öğrenci e-postayla bildirmişti. Hoca o derste iki farklı bakış açısı anlatmıştı: sayının kendisine kadar giden sürüm ve sayının karekökü ($\sqrt{x}$) kadar giden sürüm. Akış diyagramında değişiklik yaparken ikisinin karışımı kalmıştı.

Hatalı olan kısım, döngüden sonraki "`i` eşit mi `x`" kontrolüydü. Bu kontrol, sayının kendisine kadar giden eski sürüm için doğrudur. Döngü artık yalnız kareköke kadar gittiği için `i` hiçbir zaman `x`'e ulaşmaz. Hocanın düzeltmesi şöyleydi:

- Döngüden sonra `x % i` kontrol edilir. Sıfırdan farklıysa sayı asaldır (prime number), sıfırsa asal değildir.
- Döngü koşulundaki "küçük eşit" ifadesi "küçük" olarak kalmalıdır. Hoca, sınırın `i` ile karşılaştırılmasının "`i` eşit sınır mı" ya da "sınır artı bir mi" biçiminde de yapılabileceğini söyledi.

Hoca, akış diyagramlarında ya da kodlarda bir hata fark edildiğinde kendisine bildirilmesini, gerekli düzeltmelerin her zaman yapılacağını söyledi.

> **Ders sonrası not:** Akış diyagramı altyazıda görülmediği için koşul tam doğrulanamadı. Sınır $\lfloor \sqrt{x} \rfloor$ ise döngü koşulu `i <= sınır` (ya da `i * i <= x`) olmalıdır. Koşul `i < sınır` olursa karekökün kendisi denenmez. Python ile denendi: bu durumda 4,

9, 25 ve 49 gibi asal sayıların karesi olan sayılar asal sanılır. Hocanın "sınır artı bir" sözü, `<` kullanılacaksa sınırın $\lfloor\sqrt{x}\rfloor + 1$ alınması gerektiğine karşılık gelir.

2. Özyinelemeye Kısa Dönüş

Önceki derste problemlerin özyinelemeli (recursive) ya da yinelemeli (iterative) çözülebileceğini, aradaki farkı ve faktöriyel örneğini görmüştük. Faktöriyelde durma koşulu (base condition) `n` 1'den küçük ya da 1'e eşit olduğunda 1 döndürmekti. Diğer durumlarda fonksiyon kendini çağırıyor ve her çarpım değerini bir üst çağrıya geri gönderiyordu.

Özyinelemeli fonksiyonlar, bellekte hangi noktaya geri dönüleceğini yığın (stack) üzerinde saklar. Türkçede özyinelemeli fonksiyon dediğimiz bu yapı ne kadar derine inerse, yani fonksiyon kendini ne kadar çok çağırırsa, yığında o kadar yer kullanılır. Yığının bellekte belirli bir alanı vardır. Her çağrı bu alanı doldurduğu için sonunda yığın taşması (stack overflow) oluşabilir. Yığına alabileceğinden fazla veri yazılmaya çalışıldığında program hata üretir ve segmentation fault ile sonlanır. Hoca, aynı adı taşıyan ve birçok programlama sorununun konuşulduğu bir web sitesi olduğunu da hatırlattı.

> **Hocanın vurgusu:** Özellikle özyinelemeli işlerde yığın boyutunuzun ne olduğunu bilmeniz önemli.

Özyinelemenin kullanım alanları olarak ağaç (tree) ve graf algoritmaları ile geri izleme (backtracking) algoritmaları anıldı. Bunların örnekleri hem bu derste hem de sonraki yılın algoritma dersinde görülecek.

3. Özyinelemeli Problem 1: İki Sayıyı Toplama ile Çarpmak

Hoca bu bölümde birkaç problemi özyinelemeli olarak ele aldı. Amacının özyinelemeli çözümün verimli olduğunu göstermek olmadığını vurguladı. Amaç, bir problemi özyinelemeli olarak nasıl ele alacağımızı ve nasıl tasarlayacağımızı görmektir. Gerçekte iki sayıyı çarparken doğrudan çarpma işlemini kullanırız.

İki sayının çarpımı (product of two numbers), sayılardan birinin diğeri kadar kendisiyle toplanmasıdır:

- $x = 5$, $y = 3$ için $5 + 5 + 5 = 15$.
- $x = 50$, $y = 5$ için 50'yi 5 kez toplayınca 250.

Bu iş bir `for` döngüsüyle de yapılabilir. Özyinelemeli tasarımda dikkat edilecek bir nokta var: Toplamayı küçük sayı kadar adımda yapmak daha akıllıcadır. 50 defa 5'i toplamak yerine 5 defa 50'yi toplamak daha mantıklıdır. Bu yüzden önce bir küçük-büyük kontrolü yapılır ve gerekirse parametrelerin yeri değiştirilir.

```
int product(int x, int y)
{
    if (x < y)                      /* verimlilik: küçük sayı y olsun */
        return product(y, x);
    if (y != 0)
        return x + product(x, y - 1); /* self call */
    return 0;                       /* base condition */
}
```

İlk `if`, yalnız verimlilik için yazılmıştır. Asıl işi ikinci `if` yapar: `y` sıfırdan farklı olduğu sürece her adımda `x` toplanır ve fonksiyon `x` ile `y - 1` parametreleriyle kendini çağırır. `y` her çağrıda bir azaldığı için sonunda sıfıra ulaşır. `y` sıfır olduğunda fonksiyon kendini çağırmayı bırakır ve 0 döndürür, çünkü son adımda eklenecek bir şey kalmamıştır.

`product(50, 5)` çağrısının izi:

Çağrı	`y != 0`?	Dönen ifade
`product(50, 5)`	evet	`50 + product(50, 4)`
`product(50, 4)`	evet	`50 + product(50, 3)`
`product(50, 3)`	evet	`50 + product(50, 2)`

Çağrı	`y != 0`?	Dönen ifade
`product(50, 2)`	evet	`50 + product(50, 1)`
`product(50, 1)`	evet	`50 + product(50, 0)`
`product(50, 0)`	hayır	`0`

En alttan 0 döner, sonra her seviyede 50 eklenerek değer bir üst fonksiyona aktarılır: $0+50+50+50+50+50=250$. Program `gcc -Wall` ile derlenip çalıştırıldı: `product(5, 3) = 15`, `product(50, 5) = 250`, `product(5, 50) = 250` (parametreler yer değiştirdikten sonra).

Hoca bu örnekte özyinelemeli fonksiyonun olmazsa olmazlarını işaretledi:

- **Durma koşulu (base condition):** `y` sıfır olunca 0 döndüren kısım.
- **Kendini çağırma (self call):** `product(x, y - 1)` çağrısı.
- **Değerin yukarı aktarılması:** Her turda `x +` işlemiyle sonuç bir üst fonksiyona iletilir.

4. Özyinelemeli Problem 2: Rakamlar Toplamı

İkinci örnek, verilen bir sayının rakamlarının toplamıdır. 1234 için $1+2+3+4=10$ bulunmalıdır. Bu da yinelemeli yazılabilir, ama amaç özyinelemeli tasarımı anlamaktır.

Bu kez fonksiyona yalnız sayının kendisini göndermek yeterlidir. Önceki örnekte iki parametre de kullanıldığı için ikisi birden geçirilmişti. Durma koşulu olarak `n == 0` seçildi ve bu durumda 0 döndürülür. `else` yazmaya gerek yoktur: Koşul doğru değilse kod zaten bir sonraki satıra geçer.

```
int sumOfDigits(int n)
{
    if (n == 0)
        return 0;
    return n % 10 + sumOfDigits(n / 10);
}
```

Hoca kendini çağırma satırında hangi parametrenin geçirileceğini sınıfa sordu. Cevap `n / 10` oldu. Her turda yapılan şey, sayının birler basamağını (`n % 10`) toplama eklemek ve kalan kısmı (`n / 10`, tam sayı bölmesi) bir sonraki çağrıya göndermektir.

Çağrı	`n % 10`	Sonraki çağrı	Biriken ifade
`sumOfDigits(1234)`	4	`sumOfDigits(123)`	$4+\ldots$
`sumOfDigits(123)`	3	`sumOfDigits(12)`	$4+3+\ldots$
`sumOfDigits(12)`	2	`sumOfDigits(1)`	$4+3+2+\ldots$
`sumOfDigits(1)`	1	`sumOfDigits(0)`	$4+3+2+1+\ldots$
`sumOfDigits(0)`	-	0 döner	

`1 / 10` tam sayı bölmesinde 0 olduğu için son çağrıya 0 gider ve 0 döner. Değerler yukarı doğru iletilince $0+1+2+3+4=10$ elde edilir. Derlenen programın çıktısı da 10'dur. Hoca, durma koşulunun illa böyle yazılması gerekmediğini, başka kontrollerle başka durma koşulları da yazılabileceğini söyledi.

Hoca burada yığının çalışma ilkesini hatırlattı: Yığın LIFO (last in, first out, son giren ilk çıkar) çalışır. Bu, ilk girenin son çıktığı anlamına da gelir. 1234 gönderildiğinde çağrılar durma koşuluna kadar en aşağıya iner. Değerler oradan sırayla yukarı döner: önce durma koşulundan 0, sonra 1, sonra 2 gibi.

5. Özyinelemeli Problem 3: Palindrom Sayı

Palindrom, tersten yazıldığında düz haliyle aynı ifadeyi veren sayı ya da dizgidir (string). 2002 tersten de 2002'dir. 1234 ise tersten 4321 olur, palindrom değildir. Sayının 4 basamaklı olması gerekmez. 3 basamaklı ya da 57675 gibi daha uzun sayılar da palindrom olabilir. Aynı tanım dizgiler için de geçerlidir.

Hoca, çoğu öğrencinin bu problemi daha önce gördüğünü belirterek 3-4 dakika süre verdi ve problemi kâğıt kalemle özyinelemeli çözmeye çalışmalarını istedi. Süre sonunda çözümü kendisi anlattı.

Strateji şudur: Sayının soldan sağa değerini zaten biliyoruz. Sağdan sola hareket etseydik aynı değeri elde eder miydik? Bunun için sayının tersini özyinelemeli olarak hesaplayan bir fonksiyon yazılır. Hoca burada C'nin bir özelliğinden yararlandı: Toplam değişkeni `static` tanımlanır. `static` değişken fonksiyonun her çağrısında sıfırlanmaz, değeri nerede kaldıysa oradan devam eder.

```
int reverse(int number)
{
    static int sum = 0;
    int tmp;
    if (number != 0) {
        tmp = number % 10;
        sum = sum * 10 + tmp;
        return reverse(number / 10);
    }
    return sum;
}
```

`reverse(2002)` çağrısının izi (derlenen programın çıktısıyla aynıdır):

Çağrı	`number != 0`?	`tmp`	`sum`
`reverse(2002)`	evet	2	$0 \cdot 10 + 2 = 2$
`reverse(200)`	evet	0	$2 \cdot 10 + 0 = 20$
`reverse(20)`	evet	0	$20 \cdot 10 + 0 = 200$
`reverse(2)`	evet	2	$200 \cdot 10 + 2 = 2002$
`reverse(0)`	hayır	-	`return sum` → 2002

Son adımda `2 / 10 = 0` gönderilir. `number` sıfır olduğu için `return sum` çalışır ve 2002 döner. Bir öğrencinin hatırlattığı gibi bu fonksiyon yalnız sayının tersini hesaplar. Ana programda dönen sayı orijinal sayıyla karşılaştırılır: Aynıysa "palindrom", değilse "palindrom değil" yazdırılır.

Soru: LIFO'ya göre değerlerin en alttan geriye toplanması gerekmiyor muydu? Sayı 0 olunca `return sum` dedik, orada 0 dönmüyor mu? **Cevap:** Değer her çağrıda `static` olan `sum` değişkeninde biriktiği için en alttaki çağrı doğrudan biriken toplamı döndürür. Hoca, `static int` tanımının bu çözümde kilit nokta olduğunu söyledi.

> **Ders sonrası not:** `static` değişken her çağrıda sıfırlanmaz, ama program boyunca da hiç sıfırlanmaz. Aynı programda `reverse` ikinci kez çağrılırsa `sum` önceki sonuçtan devam eder. Derlenen programda `reverse(2002)` çağrısından sonra `reverse(1234)` çağrıldığında 4321 yerine 20024321 döndü. Fonksiyon birden çok kez kullanılacaksa `sum` her kullanımdan önce sıfırlanmalı ya da ikinci bir parametre olarak geçirilmelidir.

6. Özyinelemeli Problem 4: Asallık Testi

Dördüncü örnek, bilinen bir problemin özyinelemeli kurgusudur: Bir sayının asal olup olmadığını özyinelemeli bulmak. Fonksiyon iki parametre alır: test edilen sayı `n` ve denenen bölen `i`.

```
int isPrime(int n, int i)
{
    if (n <= 2)
        return (n == 2) ? 1 : 0;
    if (n % i == 0)
        return 0;
    if (i * i > n)
        return 1;
```

```
    return isPrime(n, i + 1);
}
```

Fonksiyonun parçaları şöyledir:

- **İlk durma koşulu:** `n <= 2` ise, `n` 2'ye eşitse 1, değilse 0 döner. Hoca bu koşullu ifade (`? :`) yazımının C'den bilinmesi gerektiğini söyledi.
- **Bölünebilme kontrolü:** `n % i == 0` ise sayı bölünebiliyordur, otomatik olarak asal değildir ve 0 döner.
- **Karekök kontrolü:** `i * i > n` ise kareköke kadar bölen bulunmamıştır, sayı asaldır ve 1 döner.
- **Kendini çağırma:** Hiçbir koşul sağlanmazsa `isPrime(n, i + 1)` çağrılır.

Kendini çağırma satırındaki parametreyi hoca sınıfa sordu. Bir öğrenci sohbetten `n, i + 1` cevabını yazdı. Her `if` içinde `return` olduğu için `else if` yazmaya gerek yoktur.

Ana fonksiyondan ilk çağrıda `i` için 2 gönderilmelidir, 3 değil. Bu kod böyle yazıldığı için 2 ile başlamak gerekir. 3 ile başlanırsa sayının 2'ye bölünüp bölünmediği hiç kontrol edilmez. Kod farklı tasarlanırsa 3'ten de başlanabilir. `n` kullanıcının girdiği ve her seferinde değişebilen değerdir, 2 ise sabit bir başlangıç değeridir.

`isPrime(11, 2)` izi:

Çağrı	`n <= 2`?	`n % i`	`i * i > n`?	Sonuç
`isPrime(11, 2)`	hayır	1	$4 > 11$ hayır	`isPrime(11, 3)`
`isPrime(11, 3)`	hayır	2	$9 > 11$ hayır	`isPrime(11, 4)`
`isPrime(11, 4)`	hayır	3	$16 > 11$ evet	1 döner

Dönen 1 değerinden 11'in asal olduğu anlaşılır ve ana program ilgili cümleyi ekrana yazdırır. `isPrime(12, 2)` çağrılsaydı daha ilk adımda `12 % 2 == 0` koşuluna girilir ve 0 döner. Fonksiyon kendini hiç çağırmadan, yalnız bir kez çalışarak sayının asal olmadığını bildirir. Derlenen program iki sonucu ve çağrı sayılarını doğruladı.

Bu çözüm kareköke kadar bütün sayıları dener. Hoca, iteratif çözümde daha sofistike bir sürüm yazdıklarını hatırlattı. Aynı fikir özyinelemeli sürüme de uygulanabilir: Önce 2'ye bölünebilme kontrol edilir. Bölünmüyorsa `i + 2` ile 3, 5, 7 diye ilerlenir. Böylece 4'e, 6'ya bakmaya gerek kalmaz ve daha az karşılaştırma yapılır.

7. Alıştırma: Bir Dizinin r'li Kombinasyonları

Aradan önce hoca özyineleme üzerine bir alıştırma verdi. n elemanlı bir dizi ve bir r değeri veriliyor. Dizinin elemanlarının bütün r'li kombinasyonları özyinelemeli bir fonksiyonla ekrana yazdırılmalı ve C kodu yazılmalı. Kolaylık için dizinin elemanları farklıdır (distinct, unique), yani tekrar etmez. Tekrar eden elemanlar için de çözüm üretilebilir. Elemanlar 1, 2, 3, 4, 5 olmak zorunda değildir; 5, 7, 12, 28, 92, 89 da olabilir. r de 2 ya da 4 olabilir. Önemli olan, dizinin ve r'nin parametre olarak verildiği genel bir çözümdür.

Dersin sonunda bir öğrencinin isteği üzerine alıştırma yeniden açıklandı. Kombinasyonda sıra önemli değildir. 1, 2, 3 yazıldıysa 3, 2, 1 ayrıca yazılmaz. Yazdırılan kümelerin sıralı olma zorunluluğu da yoktur. Önemli olan bütün kombinasyonların ekrana yazılmasıdır.

> **Ders sonrası not:** Hoca $\{1, 2, 3, 4, 5\}$ ve $r = 3$ için önce yalnız 6 kombinasyon yazdı: 123, 124, 125, 234, 235, 345. Dersin sonunda öğrenciler eksikleri hatırlattı. Doğrusu $\binom{5}{3} = 10$ kombinasyondur: 123, 124, 125, 134, 135, 145, 234, 235, 245, 345. İlk listede 134, 135, 145 ve 245 eksikti (python ile doğrulandı). Dersin sonunda 4 ile başlayan kombinasyonların ayrıca yazılıp yazılmayacağı da soruldu. Hoca kontrol ettikten sonra, 4, 1, 2'nin 1, 2, 4 ile aynı kombinasyon olduğunu, bu yüzden ayrıca yazılmadığını söyledi.

Alıştırmanın çözümü derste verilmedi. Bir çözüm "Örnek Sorular" bölümündedir.

8. Özyineleme Tasarımının Özeti

Aradan dönüşte hoca özyinelemeli fonksiyonlar konusunu özetledi. Bir problemi özyinelemeli hesaplamak için:

- Bir durma koşuluna (base condition), yani durma noktasına ihtiyaç vardır.
- Kendi kendini çağıran bir fonksiyon vardır.
- Bu çağrıda bir parametre aktarımı yapılır.

  Hocanın vurgusu: Parametreyi olduğu gibi kendisine aktarmak durma koşuluna ulaşmak için yeterli değildir. Parametreyi mutlaka değiştirmeniz beklenir.

Değişiklik tasarıma göre artırma ya da azaltma yönünde olabilir, ama genelde azaltma yönündedir: bir bir azaltmak, 10'a bölerek indirgemek gibi. Bazı durumlarda daha fazla işlem yapılır, ya da kendini çağıran fonksiyonun sonucu bir toplama ya da çarpma işlemine sokulur.

9. Diziden Bağlı Listeye

Hoca bağlı listeyi (linked list) temel bir veri yapısı olarak tanıttı ve önce bilinen diziyle karşılaştırdı.

Dizi, aynı tipteki verileri bellekte kesintisiz biçimde arka arkaya saklar. Hocanın örneğinde `int` tipindeki `A` dizisinin sıfırıncı elemanı 1000 adresinde, birinci elemanı 1004 adresindedir ve kaç eleman varsa bu şekilde arka arkaya yerleşir. Bunun adı kesintisiz bellek kullanımıdır (contiguous memory usage). Hoca bu terimin önemli olduğunu söyledi. Dizinin sağladığı en büyük kolaylık, adresi vererek verilere doğrudan erişmektir. İstenen veriye ulaşmak için diğer verilerin üzerinden geçmek gerekmez.

Dizinin dezavantajları da vardır:

- **Kesintisiz yer sorunu:** Büyük bir veriyle çalışıyorsanız ve bellekte arka arkaya yeterli boş yer yoksa, veriyi belleğe parça parça almak için özel işlemler yapmanız gerekir.
- **Ekleme ve silme maliyeti:** Araya bir eleman eklemek için diğer elemanları kaydırmak gerekir. Aradan bir eleman silindiğinde de diğer elemanlar kaydırılır.

Bağlı liste ise bellekte tek bir boş alan bile varsa bir elemanı oraya yerleştirebilir. Elemanlar belleğin farklı noktalarına dağınık biçimde konur. Bir bakıma diziyi dağıtık olarak saklamayı sağlar. Bir milyon elemanınız varsa ve bellekte arka arkaya bir milyonluk boşluk yoksa, elemanlar uygun boşluklara yerleştirilir ve bellek çok daha verimli kullanılır. Hocaya göre bağlı listenin en önemli avantajı budur.

Elemanlar arka arkaya durmadığı için, bir elemandan sonra hangi elemanın geldiği bilgisi mutlaka saklanmalıdır. Diğer elemanın adresini bilmeden listeye sırayla erişmek mümkün değildir. Bu yüzden her eleman hem kendi değerini hem de kendisinden sonra gelen elemanın adresini tutar. Bu bir dezavantajdır: Diziye göre her eleman için bir de adres bilgisi saklanır.

Hocanın çizdiği örnekte liste 1 → 5 → 3 → 2 → 4 biçimindeydi: 1 5'i, 5 3'ü, 3 2'yi, 2 4'ü gösteriyordu. 4'ün son eleman olduğu, gösterdiği `NULL` adresinden anlaşılır.

> **Hocanın vurgusu:** Listenin son elemanı mutlaka `NULL`'u göstermelidir. Liste oluşturulurken buna dikkat edilmelidir.

Son elemanın adres alanında başka bir adres olursa o adrese gidilir. `NULL` kullanılmadığı sürece elemanlar arasında atlamaya devam edilir ve liste hiç bitmez. Dizgilerin sonu daha önce `'\0'` karakteriyle anlaşılıyordu. Bağlı listenin sonu da `NULL` işaretçi (null pointer) ile anlaşılır: Adres sıfırı gösteriyorsa o eleman son elemandır.

Listenin baş elemanının adresi, yani `head`, mutlaka bilinmelidir. Diğer elemanlara ancak oradan başlayarak ulaşılır. Bazı uygulamalarda son elemanın adresi de tutulur. Buna `tail` (kuyruk) denir. Bağlı listede bir milyon eleman varsa, sondaki elemana ulaşmak için bir milyon elemanın hepsi ziyaret edilmelidir. Dizide ise son elemanın adresi verilerek doğrudan erişilir. Hocaya göre aradaki en önemli fark budur. `tail` tutmak bazı uygulamalarda bu işi kolaylaştırır.

10. Bağlı Listeyi Gerçekleştirme Yolları

10.1 Bağlı Liste Türlerine İlk Bakış

Hoca türleri kısaca saydı (ayrıntısı 15. bölümdedir):

- **Tek yönlü bağlı liste (single linked list):** Tek yönde ilerlenir, yalnız sonraki elemanın adresi saklanır.
- **Çift yönlü bağlı liste (double linked list):** Önceki elemanın adresi de saklanır, ileri ve geri gidilebilir. Ek bir maliyettir, ama ileri geri gezinmeyi kolaylaştırır.
- **Dairesel bağlı liste (circular linked list):** Son elemandan yeniden başa dönülebilir. Tek yönlü ya da çift yönlü kurulabilir.

10.2 İki Dizi ya da Matris ile

C ve benzeri dillerde genellikle bir `struct` yapısı kullanılır. Ama bağlı liste yalnız böyle kurulmaz. İki dizi ya da iki satırlı veya iki sütunlu bir matris de kullanılabilir. Bir dizi elemanların değerlerini, diğer dizi her elemandan sonra gelen elemanın dizinin hangi gözünde olduğunu saklar. Örneğin birinci eleman beşinci gözü gösteriyorsa sonraki eleman beşinci gözdeki elemandır. O eleman da iki numaralı gözü gösteriyorsa ondan sonraki eleman ikinci gözdedir.

Bu yöntem pek sık kullanılmaz. Çünkü diziyi tercih etmeme sebebimiz zaten arka arkaya yeterli alanın olmamasıdır. İki dizi kullanarak bağlı liste yapmak bu açıdan çok anlamlı değildir.

10.3 struct ile

En çok kullanılan ve verimli yöntem bir düğüm (node) yapısı tanımlamaktır:

```
struct node {
    int value;
    struct node *next;
};

struct node *head = NULL;
```

`value` alanı temsilidir. Tipi ne saklanıyorsa o olur. Tam sayı, bir isim ya da bir öğrencinin bütün bilgilerini tutan başka bir `struct` olabilir. Önemli olan, düğümün bir sonraki düğümün adresini tutan alanıdır. Bu alan kendi tipine işaret ettiği için `struct node *next` biçiminde yazılır. Buna kendine referans (self-referencing) denir. Hoca, burada `typedef` kullanılamayacağını, çünkü tip tanımının önce yapılması gerektiğini söyledi.

> **Ders sonrası not:** Hoca "burada `typedef` kullanamazsınız" dedi. Doğrusu: `typedef` kullanılabilir (`typedef struct node { int value; struct node *next; } Node;`). Ama `struct`'ın içindeki kendine işaret eden alan için `struct node *` yazılmalıdır, çünkü `typedef` adı o noktada henüz tanımlı değildir.

Çift yönlü liste gerekseydi `struct node *prev;` biçiminde önceki elemanı gösteren bir adres alanı daha eklenirdi. Bu, tek bir değer saklarken buna ek olarak iki adres saklamak demektir. Saklanan bilgi miktarının büyümesi bağlı listenin önemli zayıflıklarından biridir.

Listenin nereden başladığını gösteren bir başlangıç düğümü adresi, yani `head` gerekir. Her yeni düğüm için yer ayıran bir `createNode` fonksiyonu yazılır:

```
struct node *createNode(void)
{
    struct node *p;
    p = (struct node *)malloc(sizeof(struct node));
    if (p == NULL) {
        printf("Bellek ayrilamadi\n");
        exit(1);
    }
    return p;
}
```

Fonksiyon önce `struct node *p` ile bu tipte bir işaretçi tanımlar. Sonra `malloc` ya da `calloc` ile düğümün büyüklüğü kadar yer ayırır. Düğüm bir öğrencinin bilgilerini tutuyorsa o büyüklükte alan açılır. `NULL` dönmezse sorun yoktur ve `p` döndürülür. Fonksiyon `p`'yi döndürdüğü için dönüş tipi `struct node *`'dır. `NULL` dönerse bellekte tek bir eleman için bile yer kalmamıştır. Bu durumda kullanıcıya hata verilmelidir.

Hoca bunu diziyle karşılaştırdı. Dizide genellikle kullanıcıdan bir boyut alınır (bin, yüz bin, bir milyon) ve dinamik bellek ayırma ile çalışma zamanında (runtime) yer açılmaya çalışılır. Sorun, o kadar arka arkaya alanın aynı anda bellekte bulunmayabilmesidir. 999 bin değerle iş yapılabilecekken bir milyonluk alan bulunamadığı için hiçbir iş yapılamaz hale gelinebilir. Bağlı listede ise tek bir eleman için bile yer açılamadığında bir önlem alınması, belki yeni bir bellek alanı ayrılması ya da boşaltılması planlanır.

11. Gezinme, Arama ve Güncelleme

11.1 printList

`printList` fonksiyonu listedeki bütün elemanların değerlerini ekrana yazdırır. Fonksiyona başlangıç düğümünün adresi gönderilir.

```
void printList(struct node *head)
{
    struct node *p;
    for (p = head; p != NULL; p = p->next)
        printf("%d ", p->value);
}
```

`for` döngüsünde `p`'ye başlangıç adresi atanır. `p` `NULL`'dan farklı olduğu sürece, yani son elemanı geçmediğimiz sürece, `p`'nin gösterdiği düğümün `next` adresi `p`'ye atanır ve her adımda düğümün değeri yazdırılır. Daha önceki döngülerde `n`'e kadar gidiliyor ve her turda `i = i + 1` yazılıyordu. Burada ilerleme `p = p->next` ile yapılır. Aynı iş `while` ile de yazılabilir. Döngüye bir sayaç eklenirse listedeki eleman sayısı da bulunur. Fonksiyon sayaçla birlikte derlenip denendi: 3 → 5 → 9 listesi için 3 eleman sayıldı.

Soru: Bir diziyi fonksiyona parametre olarak verdiğimizde, fonksiyon içinde dizinin kaç elemanlı olduğunu bulmak için `NULL` işaretçiye ulaşana kadar döngü kurabilir miyiz? **Cevap:** Hoca önce dizinin sonunda `NULL` işaretçi olup olmadığını sordu. Bağlı listede elemanların değerleriyle ve sonraki elemanın adresiyle saklandığını, son elemanın `NULL`'u gösterdiğini hatırlattı. Bağlı listedeki eleman sayısının ayrıca tutulup tutulmayacağı duruma göre değişir. `NULL`'a ulaşana kadar elemanlar sayılarak eleman sayısı bulunabilir, bunun hiçbir sakıncası yoktur.

> **Ek bilgi (derste anlatılmadı):** Sıradan bir C dizisinin sonunda `NULL` işaretçi ya da bir bitiş işareti yoktur (dizgilerdeki `'\0'` dışında). Bu yüzden bir dizi fonksiyona geçirildiğinde eleman sayısı genellikle ayrı bir parametreyle gönderilir. `NULL`'a kadar sayma yöntemi bağlı listeye özgüdür.

Soru: `p`'yi ne kadar ilerleteceğimizi kendimiz mi yazmalıyız? Örneğin `uint8` tanımladıysam her seferinde o kadar mı ileri gideceğim, yoksa `sizeof` ile mi hesaplamalıyım? **Cevap:** Bir `struct` tanımlandığında, örneğin 30 byte'lık bilgi saklanıyorsa, her düğüm 30 byte'tır. Ama bu düğümler belleğin farklı yerlerindedir. `p = p->next` dendiğinde C, `p`'yi doğrudan `next`'in gösterdiği adrese ayarlar. O adresten itibaren `p->value`, öğrenci adı, soyadı, notu gibi `struct`'ta ne tanımlandıysa hepsine erişilebilir. `sizeof` ile uğraşmaya gerek yoktur.

11.2 Arama ve Güncelleme

`printList` aslında listede gezinmedir (traverse). Gezinirken arama (search) da yapılabilir: Döngüye bir `if` eklenerek elemanın aranan eleman olup olmadığına bakılır. İki satırlık bir eklemeyle arama tamamlanır. Fonksiyonun adı değiştirilip bir `update` fonksiyonu yazılabilir. Fonksiyona aranan eleman ve yeni değer de gönderilir, yani üç parametre olur. Bulunan elemanın değeri güncellenir. Arama ve güncellemenin gezinme kodundan hemen hemen hiçbir farkı yoktur. Ekleme (insert) ve silme (delete) ise diğer iki temel işlemdir.

Hoca, arama ve güncellemenin dizide mi bağlı listede mi daha verimli olduğunu sordu. Bir öğrencinin cevabı doğruydu: Dizi daha verimlidir, çünkü adresi söyleyerek doğrudan erişim (direct access) vardır. Bağlı listede elemanın kaçıncı eleman olduğu bilinmediği için bütün elemanları gezmek gerekebilir. Eleman başta, ortada ya da sonda olabilir. Bu yüzden arama ve güncellemenin maliyeti bağlı listede diziye göre daha fazladır.

> **Hocanın vurgusu:** Küçük boyutlu verilerle çalışıyorsak bağlı listeyi tercih etmek için bir motivasyonumuz kalmaz. Bağlı liste özellikle büyük boyutlu verilerde ve ekleme-silmenin çok yapıldığı işlerde öne çıkar.

12. Ekleme (Insert) İşlemi

12.1 Üç Durum

Hocanın örneğinde sıralı bir 3 → 5 → 9 listesi vardı. `head` 3'ü, yani 1000 adresini gösterir. 3'ün adres alanında 5'in adresi (1200) vardır. Şekildeki sayıların bir kısmı adres, bir kısmı saklanan bilgidir. Yeni bir elemanın gelebileceği üç yer vardır:

- **Başa ekleme (2 geliyor):** Yeni eleman ilk elemandan küçüktür, `head` onu göstermelidir. Yeni düğüm 1700 adresinde oluşturulmuştur ve `next` alanı `NULL`'dur. `head`'deki 1000 değeri yeni düğümün `next` alanına yazılır. `head`'e yeni düğümün adresi 1700 yazılır. Böylece 2, 3'ü gösterir.
- **Araya ekleme (4 geliyor):** Önce 4'ün yerleşeceği konum bulunur. `head`'den başlanır: 4 2'den büyük, devam. 4 3'ten büyük, devam. 4 5'ten büyük değil, yer bulundu. Yeni düğüm 1450 adresindedir ve `next`'i `NULL`'dur. 3 ile 5 arasındaki bağ kırılır ve yeni bir bağ kurulur: 3 artık 1200'ü değil 1450'yi gösterir. 1450'deki yeni düğüm de 3'ün daha önce gösterdiği 1200'ü gösterir. Böylece 3 → 4 → 5 olur.
- **Sona ekleme (12 geliyor):** 12; 2, 3, 4, 5 ve 9'dan büyüktür. 9'dan sonra eleman yoktur. Büyük olduğu sürece ilerlendiği için ayrıca `NULL`'a gelinip gelinmediği de kontrol edilmelidir. Son eleman artık 9 değil, 1300 adresindeki yeni düğümdür. 9'un `NULL` olan `next` alanına 1300 yazılır. Yeni düğümün `next`'i zaten `NULL` olduğu için başka bir şey yapılmaz.

Hoca araya eklemenin önemine dikkat çekti: Liste ne kadar uzun olursa olsun ekleme için yalnız birkaç işlem yapılır. Aradaki bağ kırılır, önceki elemanın adres alanı yeni elemana, yeni elemanın adres alanı da önceki elemanın gösterdiği düğüme bağlanır. Hiçbir şey kaydırılmadan, hiçbir şeyin yeri değiştirilmeden ekleme çok hızlı yapılır.

12.2 Akış Diyagramı ve Kod

Hoca ardından ekleme işleminin akış diyagramını anlattı. Bir düğüm için yer ayrılır (`p`) ve eklenecek değer verilir. Kullanıcıdan alınan değer yeni düğümün değerine atanır (`p->value = value`). Sonra adımlar şöyledir:

- **Liste boş mu?** `head == NULL` ise listede hiç eleman yoktur. Örneğin 2, 3, 5, 7, 9'dan oluşan bir diziden bağlı liste oluşturuluyor ve ilk eleman 2 geliyor. Bu durumda `head` artık `p`'yi gösterir.
- **Başa mı gelecek?** Liste boş değilse `p->value < head->value` diye bakılır. Örneğin 1 gelirse 1 < 2 olduğu için `p->next = head` ve `head = p` yapılır. `head` yeni düğümü, yeni düğüm de `head`'in eskiden gösterdiği düğümü gösterir.
- **Araya ya da sona:** Küçük değilse uygun yer bulunana kadar ilerlenir. Önceki adresi kaybetmemek için iki değişken tutulur: `prev = head` ve `current = head->next`. `current != NULL` (son elemana gelmedik) ve `current->value < p->value` (yeni değer o anki değerden büyük) olduğu sürece `prev = current` ve `current = current->next` yapılır. Döngüden çıkınca `prev->next = p` ve `p->next = current` atanır.

Bağı kırıp yeni elemanı yerleştirmek için hem o anki adresin hem de bir önceki adresin elimizde olması gerekir. Hocanın örneğinde 1 → 2 → 3 listesine 5 geldiğinde 5; 1'den, 2'den, 3'ten büyüktür ve `NULL`'a gelinip döngüden çıkılır, 5 sona eklenir. Liste 1 → 2 → 3 → 5 iken 4 gelirse 4; 1'den, 2'den, 3'ten büyüktür, 5'ten büyük değildir. Koşul bozulur ve 4 araya girer.

Derste anlatılan akışın C karşılığı:

```c
void insertSorted(int value)
{
    struct node *p, *prev, *current;
    p = createNode();
    p->value = value;
    p->next = NULL;
    if (head == NULL)
        head = p;
    else if (p->value < head->value) {
        p->next = head;
        head = p;
    }
    else {
        prev = head;
        current = head->next;
        while (current != NULL && current->value < p->value) {
            prev = current;
            current = current->next;
        }
        prev->next = p;
        p->next = current;
    }
}
```

Bu kod `gcc -Wall` ile uyarısız derlendi. 3, 5, 9 sırayla eklenip sonra 2, 4 ve 12 eklendi. Liste sırasıyla `2 3 5 9`, `2 3 4 5 9` ve `2 3 4 5 9 12` oldu. Yani üç durum da derste anlatıldığı gibi çalıştı.

Soru: Akış diyagramında `if`'in doğru kolunda `head`'e `p` atandıktan sonra `head`'in değerine yeniden `value` atamaya gerek var mı? `if`'ten önce `p->value = value` yapılmıştı, bu yeterli değil mi? **Cevap:** Hoca, `head = p` satırının adres aktarımını, alttaki satırın değer aktarımını temsil ettiğini söyledi. Adres ve değer atamasının ayrı adımlar olduğunu göstermek ve daha açık anlaşılması için yazılmıştır. Kodlarken gerektiği gibi yazılabilir.

> **Ders sonrası not:** Soru yerindedir. `p->value` daha önce atandığı ve `head` artık `p`'yi gösterdiği için `head->value = value` satırı gereksizdir. Yukarıdaki kodda bu satır yoktur.

13. Bağlı Listelerin Kullanım Alanları

Silmeye geçmeden önce hoca bağlı listelerin gerçek hayattaki kullanımlarından söz etti:

- Web tarayıcılarında sayfalar arasında ileri-geri gezinme.
- Telefondaki resim görüntüleyicide resimler arasında ileri-geri gezinme.
- Müzik uygulamalarında sonraki şarkıya geçme ya da önceki şarkıya dönme.
- İşletim sistemlerinde Alt+Tab ile uygulamalar arasında sırayla geçiş ve sonda yeniden başa dönme.

Bağlı listelerde de dizilerde olduğu gibi genellikle ekleme, güncelleme, silme ve arama işlemleri gerçekleştirilir.

14. Silme (Delete) İşlemi

14.1 Durumlar

Silme, mevcut bir bağı koparmak ve önceki elemanı sonraki elemana bağlamaktır. Hocanın örneğinde `head` 5'i, 5 9'u, 9 da 11'i gösteriyordu. Olası durumlar:

- **İlk eleman silinir:** `head` artık ikinci elemanı göstermelidir.
- **Ortadaki bir eleman silinir:** Önceki eleman, silinen elemanın gösterdiği adresi (11'in adresi 1200) göstermelidir.
- **Son eleman silinir:** Bir önceki elemanın `NULL`'u göstermesi sağlanır.
- **Silinmek istenen değer listede yoktur.**

- **Liste boştur.**

Hoca bunların hepsinin gerçek hayatta karşımıza çıkabileceğini söyledi.

Hoca bir elemanı silince bellek yönetimi için mutlaka yapılması gereken işlemi sordu. Bir öğrencinin cevabı doğruydu: İlgili alan `free` ile boşaltılmalıdır. Bağ kırılıp yeniden kurulduktan sonra (o düğümdeki bilgiler kullanıldıktan sonra) boşa çıkan düğüm C'de mutlaka `free` edilmelidir. Düğümde ne kadar bilgi olduğunu bilmeyiz, boşaltmak her zaman avantaj sağlar. Gömülü sistemlerde tek bir elemanı temizlemek bile bellek kazandırır.

> **Hocanın vurgusu:** Java'da bu iş çöp toplayıcı (garbage collector) ile yönetilir. C'de ise programcının sorumluluğundadır, unutulmamalıdır.

14.2 Akış Diyagramı

- **Liste boş mu?** `head == NULL` ise listede silinecek eleman yoktur. Kullanıcıya listenin boş olduğu söylenir.
- **İlk eleman mı?** Liste boş değilse `head`'in gösterdiği düğümün değeri silinecek değerle karşılaştırılır. Eşleşirse `tmp = head`, `head = head->next` ve `free(tmp)` yapılır. 5 siliniyorsa, 5'in adresi (600) önce `tmp`'ye alınır. `head->next` (9'un adresi 1000) `head`'e atanır ve `head` doğrudan 9'u gösterir. Sonra 600 adresi boşaltılır. Adres `tmp`'de saklanmazsa `head` güncellendiği anda kaybolur ve 5'i bellekten kaldırma imkânı ortadan kalkar.
- **Değilse döngü:** Aranan eleman ilk eleman değilse arkasında ne olduğu bilinmez, tek bir `if` yetmez, döngü gerekir. Yine önceki ve o anki adres gerekir: `prev = head`, `current = head->next`. `current != NULL` ve `current->value != value` olduğu sürece `prev = current`, `current = current->next` yapılır.
- **Döngüden sonra:** İki durum vardır. `current == NULL` ise listenin sonuna gelinmiştir, eleman listede yoktur ve "not found" yazılır. Değilse eleman bulunmuştur: `prev->next = current->next` yapılır. 9 siliniyorsa 5, 9'un gösterdiği 1200'ü gösterir ve 11'e bağlanır.

Hoca akış diyagramını tamamladığında bir öğrenci `current`'ın da `free` edilmesi gerekmediğini sordu. Hoca haklı olduğunu söyledi ve yerini öğrenciye sordu. Cevap: `prev->next = current->next` atamasından sonra `free(current)`. Böylece 9'u taşıyan düğümün kapladığı yer başka kullanımlar için serbest kalır.

```
void deleteNode(int value)
{
    struct node *tmp, *prev, *current;
    if (head == NULL) {
        printf("Liste bos\n");
        return;
    }
    if (head->value == value) {
        tmp = head;
        head = head->next;
        free(tmp);
        return;
    }
    prev = head;
    current = head->next;
    while (current != NULL && current->value != value) {
        prev = current;
        current = current->next;
    }
    if (current == NULL)
        printf("not found\n");
    else {
        prev->next = current->next;
        free(current);
    }
}
```

Kod derlenip 5 → 9 → 11 listesinde denendi. 7 silinmek istendiğinde “not found” yazdı ve liste değişmedi. 9 silinince 5 11, 5 silinince 11, 11 silinince boş liste kaldı. Boş listede silme denenince “Liste bos” yazdı.

14.3 Sıralı ve Sırasız Liste

Hoca şimdiye kadarki örneklerin sıralı olduğunu, ama bağlı listenin sıralı (sorted) da sırasız (unsorted) da olabileceğini söyledi. Yukarıdaki döngü koşulu (`!=`) sırasız liste varsayımıyla yazılmıştır. Liste sıralıysa koşula küçüktür (`<`) yazılır ve döngüden çıktıktan sonra değerin eşit olup olmadığı ayrıca kontrol edilir. Sıralı listede silinecek eleman geçildiyse geri kalanlara bakmaya gerek yoktur. Aynı durum dizilerde aramada da geçerlidir.

14.4 Silmeyle İlgili Sorular

Soru: `head`’in içinde hem değer hem adres mi tutuluyor, yalnız adres mi? **Cevap:** Genellikle `head`’de yalnız adres tutulur.

Soru: O zaman akış diyagramında `head`’in değerini nasıl kontrol ettik? **Cevap:** Hoca, sorunun yerinde olduğunu ve diyagramın kafa karıştırabileceğini söyledi. Bakılan şey `head`’in gösterdiği adresteki düğümün değeridir (`head->value`). Aşağıda `head->next` de aynı şekilde okunur. `head` bir adrestir. `p` de bir adrestir, ama `p` yeni oluşturulan düğümün değerlerine erişmek için kullanılır. `head` üzerinden de bu değerlere dolaylı olarak ulaşılır.

Soru: İlk eleman silinirken `tmp` değişkenini hiç oluşturmadan yalnız `head = head->next` yapsaydık olmaz mıydı? **Cevap:** `tmp` olmasaydı `head`’in gösterdiği adres bir yerde saklanmazdı. `head`’e `head->next` atandığı anda ilk değer kaybolurdu. Oysa silinmek istenen, `head`’in gösterdiği 5 değerli düğümdür. Silme kaygısı olmasaydı `tmp`’ye gerek yoktu.

Soru: Arama kısmı ayrı bir alt yordam (subroutine) olarak yazılsaydı işlemleri ifade etmek daha kolay olmaz mıydı? **Cevap:** Yapılabilir, böyle bir zorunluluk yok, ama mantıklı olup olmadığına bakmak gerekir. Ekleme ve silmenin dinamikleri birbirinden biraz farklıdır. İkisindeki gezinme kısmı çok benzer ve bir fonksiyon olarak yeniden kullanılabilir hale getirilebilir. Diğer kısımlar birbirinden farklıdır. Sonuçta ikisinde de liste belirli bir koşula göre gezildiği için belli bir düzenleme yapılabilir.

15. Bağlı Liste Türleri

Arama, güncelleme, ekleme ve silme bu mantıkla yapılır. Ama bu tek yönlü listedir. Çift yönlü, dairesel ve dairesel çift yönlü listelerde gerçekleştirme biraz değişir.

15.1 Tek Yönlü Bağlı Liste

Her eleman sonraki elemanı gösterir ve bu `NULL` görülene kadar sürer. Elemanlara ulaşmak için birbirlerinin üzerinden geçmek gerekir. Geri dönüş yoktur, yalnız ileri gidilir. Tek yönlü listenin dezavantajı budur. Örneğin müzik uygulamasında önceki şarkıya dönebilmek için uygulama o şarkının konumunu saklar. Tek yönlü listede böyle bir imkân yoktur. Aynı durum tarayıcı için de geçerlidir.

15.2 Çift Yönlü Bağlı Liste

Her düğümde bir değer (data) ve iki adres vardır. İlk elemanın `prev`’i `NULL`’u, son elemanın `next`’i `NULL`’u gösterir. `head` ilk elemanı gösterir. Elemanlar arasında ileri ve geri hareket edilebilir.

Ortadan bir eleman silinirken birden fazla güncelleme gerekir. Silinen elemanın önceki düğümünün `next`’i, silinen elemanın sonraki düğümünü göstermelidir. Sonraki düğümün `prev`’i de silinen elemanın önceki düğümünü göstermelidir. Araya eleman eklerken de yeni elemanın `prev`’i önceki elemanı, önceki elemanın `next`’i yeni elemanı gösterir. Yeni elemanın `next`’i sonraki elemanı, sonraki elemanın `prev`’i yeni elemanı gösterir. İş biraz karışır ama zor değildir.

> **Hocanın vurgusu:** Çift yönlü bağlı listede ekleme, silme, güncelleme ve aramayı kendi kendinize yazmaya çalışmanızı alıştırma olarak öneririm. Biraz daha zordur ama konuyu anlamanız için müthiş fayda sağlar.

Çift yönlü listede daha fazla bilgi, en azından bir adres daha saklanır. Hoca bunun hesabına dikkat çekti: Adres eklemek saklanan bilgiyi her zaman iki ya da üç katına çıkarmaz. Değer 40 byte'lık bir bilgiyse ve adres 4 byte ise tek yönlü listede düğüm 44 byte, iki adresle 48 byte olur. Saklanan bilgi yalnız 4 byte olsaydı, önceki ve sonraki adresle birlikte $4+4+4=12$ byte, yani 3 kat olurdu. Saklanan bilginin boyutu değiştiği için çift yönlü liste "3 kat bilgi saklamak" demek değildir, yalnız iki adres bilgisinin eklenmesi demektir.

> **Ek bilgi (derste anlatılmadı):** Adresin boyutu sisteme bağlıdır. Bu notu hazırlarken kullanılan 64 bitlik sistemde `sizeof(struct node *)` 8 byte çıktı. `int value` ve `next`'ten oluşan `struct node` ise 4 + 8 = 12 değil 16 byte çıktı, çünkü derleyici işaretçiyi hizalamak için araya dolgu (padding) koyar. Gerçek boyut `sizeof` ile ölçülmelidir.

Hoca ayrıca `head`'in her zaman ilk elemanın adresini gösterdiğini, `tail` varsa son elemanın adresini gösterdiğini hatırlattı. Adres, `prev` ya da `next` demek değildir, o elemanın bellekte bulunduğu adres demektir. Örneğin ilk eleman 300 adresindeyse `head` 300'ü gösterir. Bu elemanın `prev`'i `NULL`, `next`'i ikinci elemanın adresidir.

15.3 Dairesel Bağlı Liste

Dairesel bağlı liste (circular linked list) işletim sistemi seviyesinde, özellikle süreç zamanlamada (process scheduling) kullanılır. Her süreç işlemciden bir süre alır. Süre bitince işi dondurulur ve zamanlayıcı (scheduler) bir sonraki süreci seçer. Listedeki süreçler bitince ilk sürecin yeniden hizmet alması için başa dönmek gerekir. Dairesel liste bu işi çok kolaylaştırır.

Tek yönlü listeden tek farkı, son elemanın `next`'inin `NULL` yerine ilk elemanın adresini göstermesidir. Son elemanın `next`'i ile `head`'in gösterdiği adres aynıdır. Hocanın örneğinde ikisi de 500'ü gösteriyordu.

15.4 Dairesel Çift Yönlü Bağlı Liste

Çift yönlü listeye ek olarak son elemanın `next`'i `head`'in gösterdiği ilk elemanı gösterir. İlk elemanın `prev`'i de son elemanın adresini (örnekte 900) gösterir. Hocaya göre en çok kullanılan bağlı liste türleri bunlardır.

15.5 Dairesel Listede Sonu Anlamak

Hoca dairesel listelerde listenin bittiğinin nasıl anlaşılacağını sordu ve "`head` referansımız olmalı" ipucunu verdi. Bir öğrenci, bir eleman aranırken son elemana gelindiğinde hâlâ bulunamadıysa başa mı dönüleceğini sordu. Hoca bunun da bir durum olduğunu söyledi. Tek yönlü listede `NULL`'a kadar kontrol yapılıyordu. Dairesel listede adresler tekil (unique) olduğu için aynı adreste başka bir düğüm bulunamaz. Bu yüzden arama, `head`'in adresine gelinene kadar, yani `current->next == head` olana kadar sürdürülür. Hoca bunun, farklı türdeki listelerde tasarımda değişiklik gerektiğini gösteren güzel bir örnek olduğunu söyledi.

> **Ek bilgi (derste anlatılmadı):** Dairesel listede `NULL` kontrolüyle arama yapılırsa ve aranan değer listede yoksa döngü hiç bitmez. Aşağıdaki gibi bir `do-while` döngüsü, ilk düğümü de kontrol ettikten sonra başa dönünce durur. Bu fonksiyon $1 \to 2 \to 3 \to 1$ dairesel listesinde derlenip denendi: 3 bulundu, 7 için `NULL` döndü.

```c
struct node *searchCircular(struct node *head, int value)
{
    struct node *current = head;
    if (head == NULL)
        return NULL;
    do {
        if (current->value == value)
            return current;
        current = current->next;
```

```
    } while (current != head);
    return NULL;
}
```

15.6 Sonraki Konulara Bağlantı

Hoca bir sonraki konunun yığın (stack) olduğunu, ama bu hafta yığına geçilmeyeceğini söyledi. Önce bağlı listenin avantajları, dezavantajları ve günümüzdeki uygulamaları özetlenecek. İleride ağaçlarda inorder ve preorder gezinmelerde, graflarda derinlik öncelikli arama (depth first search) ve genişlik öncelikli arama (breadth first search) yapılırken bağlı liste yapısından nasıl yararlanıldığı da görülecek.

Örnek Sorular

Derste ekran görüntüsü alınmış kare bulunmadığı için bu bölüm yalnız derste sözlü sorulan sorulardan kurulmuştur.

1. (derste sözlü, 00:18:00) Rakamlar toplamını bulan `sumOfDigits(n)` fonksiyonunda `return n % 10 + sumOfDigits(...)` satırına hangi parametre geçirilmelidir?

> **Çözüm:** `n / 10`. Tam sayı bölmesi birler basamağını atar, $1234 \to 123 \to 12 \to 1 \to 0$. `sumOfDigits(1234) = 10`.

2. (derste sözlü, 00:24:31) Verilen bir sayının palindrom olup olmadığını özyinelemeli olarak bulun. Örnekler: 2002 palindrom, 1234 değil, 57675 palindrom.

> **Çözüm:** Sayının tersi `static int sum` kullanan `reverse` fonksiyonuyla bulunur (5. bölüm). `reverse(2002)` sırasıyla `sum` = 2, 20, 200, 2002 üretir ve 2002 döner. Ana programda `reverse(x) == x` ise sayı palindromdur.

3. (derste sözlü, 00:37:33) `isPrime(n, i)` fonksiyonu kendini çağırırken hangi parametreleri göndermelidir?

> **Çözüm:** `isPrime(n, i + 1)`. Test edilen sayı aynı kalır, denenen bölen bir artar.

4. (derste sözlü, 00:38:07) Ana fonksiyondan `isPrime` ilk kez çağrılırken `i` için hangi değer gönderilmelidir?

> **Çözüm:** 2. 3 ile başlanırsa sayının 2'ye bölünüp bölünmediği hiç kontrol edilmez. `isPrime(11, 2)` üç çağrıda 1 döndürür.

5. (derste sözlü, 00:40:55) `isPrime(12, 2)` çağrılsaydı hangi `if`'e girilirdi?

> **Çözüm:** `n % i == 0` kontrolüne girilir, çünkü $12 \bmod 2 = 0$. Fonksiyon 0 döndürür, yalnız bir kez çalışır ve kendini hiç çağırmaz.

6. (derste sözlü, 00:43:12 ve 02:18:02) n elemanlı, elemanları tekrarsız bir dizinin bütün r'li kombinasyonlarını özyinelemeli bir C fonksiyonuyla ekrana yazdırın. Örnek: $\{1, 2, 3, 4, 5\}$, $r = 3$. Sıra önemli değildir; 1, 2, 3 yazıldıysa 3, 2, 1 yazılmaz.

> **Çözüm (derste verilmedi):** Her adımda bir eleman seçilir ve kalan seçimler o elemandan sonraki elemanlar arasından özyinelemeli yapılır. `index == r` olunca, yani r eleman seçilince, seçilenler yazdırılır (durma koşulu). Döngü sınırı `n - (r - index)`, kalan seçimler için yeterli eleman kalmasını sağlar. Program `gcc -Wall` ile derlendi. $\{1, 2, 3, 4, 5\}$ ve $r = 3$ için 10 satır üretti: 123, 124, 125, 134, 135, 145, 234, 235, 245, 345. Bu, $\binom{5}{3} = 10$ ile ve python `itertools.combinations` çıktısıyla aynıdır.

```c
#include <stdio.h>

/* a[start..n-1] içinden kalan elemanları seçerek r'li kombinasyonları yazdırır */
void combination(int a[], int n, int r, int start, int data[], int index)
{
    int i, j;
    if (index == r) {              /* durma koşulu: r eleman seçildi */
        for (j = 0; j < r; j++)
            printf("%d ", data[j]);
```

```
        printf("\n");
        return;
    }
    for (i = start; i <= n - (r - index); i++) {
        data[index] = a[i];
        combination(a, n, r, i + 1, data, index + 1);   /* kendini çağırma */
    }
}

int main(void)
{
    int a[] = {1, 2, 3, 4, 5};
    int data[5];
    combination(a, 5, 3, 0, data, 0);
    return 0;
}
```

7. (derste sözlü, 01:23:13) Arama ve güncelleme işlemleri dizide mi bağlı listede mi daha verimlidir?

> **Çözüm:** Dizide. Dizide adresle doğrudan erişim vardır. Bağlı listede elemanın yeri bilinmediği için en kötü durumda bütün düğümler gezilir.

8. (derste sözlü, 01:53:27) Bağlı listeden bir eleman silindiğinde bellek yönetimi için mutlaka yapılması gereken işlem nedir?

> **Çözüm:** Boşa çıkan düğüm `free` ile serbest bırakılmalıdır. C'de bu programcının sorumluluğundadır; Java'da çöp toplayıcı yapar.

9. (derste sözlü, 02:01:53) Silme akış diyagramında aradaki düğüm için `free` nereye yazılmalıdır?

> **Çözüm:** `prev->next = current->next` atamasından sonra `free(current)`. Daha önce yazılırsa `current->next` okunamaz hale gelir.

10. (derste sözlü, 02:14:44) Dairesel bağlı listede listenin bittiği nasıl anlaşılır?

> **Çözüm:** `NULL` kontrolü işe yaramaz, çünkü son düğüm ilk düğümü gösterir. Gezinme `head`'in adresine dönülünce, yani `current->next == head` olunca durdurulur.

Dersin Toparlaması

Dersin ilk yarısında özyinelemeli tasarım dört örnekle çalışıldı: toplama ile çarpma, rakamlar toplamı, palindrom kontrolü ve asallık testi. Her örnekte aynı iskelet vardı: bir durma koşulu, parametresi değiştirilerek yapılan bir kendini çağırma ve dönen değerin yukarı aktarılması. Yığının LIFO çalışması, değerlerin neden en alttaki durma koşulundan yukarı doğru toplandığını açıklıyor. Özyinelemeyi pekiştirmek için bir dizinin r'li kombinasyonlarını yazdırma alıştırması verildi.

İkinci yarıda bağlı listeye geçildi. Dizi kesintisiz bellek ve doğrudan erişim sağlar, ama büyük verilerde yer bulma ve ekleme-silmede kaydırma sorunları vardır. Bağlı liste dağınık belleği kullanır ve ekleme-silmeyi yalnız bağları değiştirerek yapar, ama her düğüm ek adres saklar ve arama için liste gezilmelidir. Sıralı listeye ekleme ve listeden silme `prev` ve `current` işaretçileriyle akış diyagramı üzerinden işlendi, silinen düğümün `free` edilmesi vurgulandı. Son olarak tek yönlü, çift yönlü, dairesel ve dairesel çift yönlü listeler tanıtıldı. Sonraki derste bağlı listenin avantaj ve dezavantajları özetlenecek, ardından yığın (stack) ve kuyruk (queue) yapılarına geçilecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.